

Software Development Documentation

ECI_Android-SV1.7.8-(UVC)

Revision Record:

Version	Update Time	Update Details
v1.7.8	2023-10-13	Fixed the issue that TCI322 devices would not turn on
v1.7.7	2023-10-9	Fixed the issue that TCI322 devices would not turn on
v1.7.6	2023-8-1	Fix the problem of duplicate class packaging
v1.7.5	2023-7-11	Fix text display issues
v1.7.4	2023-7-4	Resolve abnormal opening of screen rotation device
v1.7.3	2023-7-3	Adapt to ECI303
v1.7.2	2023-6-6	Added support for opengl preview
v1.7.1	2023-5-18	Add support for obtaining iris center coordinates interface
v1.7.0	2023-5-9	Support ECI103 devices, add image preview flipping function
v1.6.11	2023-4-11	ECI302 adds switch light function
v1.6.10	2023-3-1	Update the algorithm to optimize the effect of ECI302 drawing
v1.6.9	2023-2-14	The problem cannot be identified by optimizing some devices
v1.6.8	2023-1-13	Acquire image to limit iris size
v1.6.7	2023-1-6	Support ECI305 devices
v1.6.6	2023-1-3	Support TCI303 devices
v1.6.5	2022-9-16	Compatible with TCI322 devices
v1.6.4	2022-7-25	The authentication method passes in the Feature object
v1.6.3	2022-7-7	Optimize collection effect
v1.6.2	2022-6-27	Adjusted focus parameter
v1.6.1	2022-6-17	Improved Iris image quality
v1.6.0	2022-5-16	Added focus setting for 30cm device
v1.5.9	2022-5-12	Fixed jar conflicts
v1.5.8	2022-5-7	Compatible with 25cm and 30cm focal length device

v1.5.7	2022-3-11	Optimized UI display
v1.5.6	2022-3-4	Rolled back to production version algorithm
v1.5.5	2022-2-25	Updated the algorithm to improve the passing rate
v1.5.4	2022-2-21	Adjusted parameter of 25cm device, add stress test tool
v1.5.3	2022-2-18	Added the focus setting parameter to compatible with different focus device
v1.5.2	2022-2-17	301MS device 25cm focus version
v1.5.1	2022-1-24	Fixed the hint of getting image
v1.5.0	2022-1-18	Added the interface to get image
v1.4.4	2021-7-5	Fixed an issue that USB device's Serial information could not be obtained when targetSdkVersion>=29
v1.4.3	2021-6-25	Added the interface to get the JP2K format image
v1.4.2	2021-5-27	Added the interface to get iris position coordinates
v1.4.1	2021-4-26	Fixed the bug of crash due to starting camera failure
v1.4.0	2021-4-21	Compatible with Android10
v1.3.1	2021-4-14	Fix the bug of crash caused by plugging and unplugging device
v1.3.0	2020-1-26	Optimized the algorithm to compatible with different devices
v1.2.1	2020-6-8	Updated demo
v1.2.0	2019-9-24	Compatible with Windows version features
v1.1.0.0	2018-10-22	Replace the new algorithm, improve speed
v1.0.0.1	2018-5-11	Added the cancel operation interface
v1.0	2017-12-22	1. Open the camera directly with UVCCamera, instead of using UVCCameraHandler 2. Fixed overflow of memory when opening multiple times 3.Fixed 'Too many files open'

Content

Software Development Documentation	1
ECI_Android-SV1.7.8-(UVC)	1
1 Introduction.....	5
1.1 Purpose	5
1.2 Scope	5
1.3 Directory Structure	5
2 Technical specifications and introduction.....	5
2.1 Development language	5
2.2 Operating Enviornment	5
2.3 Supported Devices	6
2.4 Function Introduction.....	6
3 SDK Specification.....	6
3.1 Intergration Introduction.....	6
3.1.1 Import Files	6
3.1.2 Add Permission	6
3.1.3 Initialize SDK.....	6
3.2 Typical function invocation examples.....	7
3.3 Detailed API Instruction.....	10
3.3.1 Iris algorithm interface.....	10
3.3.2 Iris Fragment.....	14
3.4 Error Code	15

1 Introduction

1.1 Purpose

This document is intended for technical person who develops using this SDK.

1.2 Scope

Use our Iris Device.

1.3 Directory Structure

Content	Description
SDK	Store libraries, and all the dependent files, model files, configure files, and sound files, mask files, etc.
Demo	Store executable demo program and source code of demo
Doc	Store documentation (Development Documentation, etc.)

2 Technical specifications and introduction

2.1 Development language

Java

2.2 Operating Enviornment

OS Version: [Android](#)

Development Environment: [Android Studio](#)

Minimum configuration recommended: [Processor:CPU 1.5GHZ or higher; Memory: 2GB or higher](#)

2.3 Supported Devices

ECI301MS、TCI303、ECI302、ECI305、TCI322

2.4 Function Introduction

Extract iris features and compare iris features

3 SDK Specification

3.1 Intergration Introduction

3.1.1 Import Files

lib-eyecooliris-uvc: Iris function library

Import *lib-eyecooliris-uvc* in AS as module

3.1.2 Add Permission

```
<manifest...
  <uses-permission android:name="android.permission.CAMERA"/>
  <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
  <uses-permission android:name="android.permission.MOUNT_UNMOUNT_FILESYSTEMS"/>
  <uses-permission android:name="android.permission.ACCESS_WIFI_STATE"/>
  <uses-permission android:name="android.permission.CHANGE_WIFI_STATE"/>
  <uses-permission android:name="android.permission.CHANGE_NETWORK_STATE"/>
  <uses-permission android:name="android.permission.INTERNET"/>
  <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
  <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>
</manifest>
```

3.1.3 Initialize SDK

```
IrisSDK irisSDK = IrisSDK.getInstance(getApplicationContext());
```

```
irisSDK.init("", new IrisSDK.IrisSDKCallback())
```

3.2 Typical function invocation examples

Preview, enrollment and verification with IrisFragment

Declare IrisFragment in the layout file. The iris camera uses 1920 * 600 pixels, so it's better to set width and height in the layout file as following.

```
<fragment
    android:id="@+id/irisFragment"
    android:name="com.eyecool.fragment.IrisFragment"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"/>
```

Enrollment

```
/**
 * Iris enroll
 *
 * @param timeout
 * @param enrollCallback
 */
public synchronized void enroll(final long timeout, final EnrollCallback enrollCallback)

public interface EnrollCallback {
    void onSuccess(byte[] template);

    void onError(int errCode, String msg);

    void onProcess(int progress, int state);
}
```

1: N Verification

```
/**
 * Iris (1: N)
 *
 * @param timeout
 * @param featureList
 * @param verifyCallback
```

```

    */
    public synchronized void verify(final long timeout, final List<Feature> featureList, VerifyCallback verifyCallback)

    public interface VerifyCallback {
        void onSuccess(int position);

        void onError(int errCode, String msg);
    }

    public interface VerifyCallbackX extends VerifyCallback {
        void onProcess(int state);
    }

```

Cancel

```

public void cancel(){
}

```

Get image

```

/**
 * Acquire Iris Image
 *
 * @param timeout
 * @param quality
 * @param imageType      0:K1, 1:VGA (K2), 2:K3, 3:K7
 * @param acquireImageCallback
 */
    public synchronized void acquireImage(final long timeout, final int quality, int imageType, final AcquireImageCallback acquireImageCallback)

    public interface AcquireImageCallback {
        void onSuccess(IrisResult result);

        void onError(int errCode, String msg);

        void onProcess(int state);
    }

```


com.eyecool.iris.api.IrisResult

```
/**
 * Get image data of left iris
 *
 * @return
 */
public byte[] getLeftImage() {
    return leftImage;
}

/**
 * Get quality score of left iris image
 *
 * @return
 */
public int getLeftQualityScore() {
    return leftQualityScore;
}

/**
 * Get diameter of left iris
 *
 * @return
 */
public int getLeftDiameter() {
    return leftDiameter;
}

/**
 * Get valid area of left iris image
 *
 * @return
 */
public int getLeftArea() {
    return leftArea;
}

/**
 * Get image data of right iris
 *
 * @return
 */
public byte[] getRightImage() {
    return rightImage;
}
```

```

}

/**
 * Get quality score of right iris image
 *
 * @return
 */
public int getRightQualityScore() {
    return rightQualityScore;
}

/**
 * Get diameter of right iris
 *
 * @return
 */
public int getRightDiameter() {
    return rightDiameter;
}

/**
 * Get valid area of right iris image
 *
 * @return
 */
public int getRightArea() {
    return rightArea;
}

```

3.3 Detailed API Instruction

3.3.1 Iris algorithm interface

com.eyecool.iris.api.IrisSDK

```

/**
 * Get instance
 *
 * @return
 */

```

```

public static IrisSDK getInstance()

/**
 * Initialization
 *
 * @param context      context
 * @param appId        APP ID
 * @param irisSDKCallback Callback
 */
public void init(Context context, String appId, IrisSDKCallback irisSDKCallback)

/**
 * Enroll data
 *
 *                                     *      @param      imageData      Image      data
(RAW format, the width and height of the image is the size of the device drawing)
 * @param state      int[2] 0:progress 1: location information
 * @return >=0 registration succeeded,<0 registration failed
 */
public synchronized int enrollData(byte[] imageData, int[] state)

/**
 * Enroll data
 *
 * @param imageData Image data (RAW format)
 * @param width      Image width
 * @param height     Image height
 * @param state      int[2] 0:progress 1: location information
 * @param pixelMin   Minimum size of registered iris
 * @param pixelMax   Maximum size of registered iris
 * @return >=0 registration succeeded,<0 registration failed
 */
public synchronized int enrollData(byte[] imageData, int width, int height, int[] state, int pixelMin, int pixelMax)

/**
 * Get feature from iris image
 *
 *                                     *      @param      imageData      imageData      Image      data
(RAW format, the width and height of the image is the size of the device drawing)
 * @return >= 0 succeeded, < 0 failed
 */
public byte[] getFeature(byte[] imageData)

```

```

/**
 * Get feature from iris image
 *
 * @param imageData Image data (RAW format)
 * @param width Image width
 * @param height Image height
 * @param feature Output feature
 * @return >= 0 succeeded, < 0 failed
 */
public int getFeature(byte[] imageData, byte[] feature)

/**
 * Get feature from iris image
 *
 * @param imageData Image data (RAW format)
 * @param width Image width
 * @param height Image height
 * @param feature Output feature
 * @return >= 0 succeeded, < 0 failed
 */
public int getFeature(byte[] imageData, int width, int height, byte[] feature)

/**
 * Get feature from iris image
 *
 * @param imageData Image data (RAW format)
 * @param width Image width
 * @param height Image height
 * @param feature Output feature
 * @param pixelMin Minimum size of iris
 * @param pixelMax Maximum size of iris
 * @return >= 0 succeeded, < 0 failed
 */
public int getFeature(byte[] imageData, int width, int height, byte[] feature, int pixelMin, int pixelMax)

/**
 * Iris feature comparison
 *
 * @param verifyFea feature of verification
 * @param template template
 * @return Comparison score
 */
public int compare(byte[] verifyFea, byte[] template)

```

```

/**
 * Convert iris image
 *
 * @param imageData (in)Original iris image
 * @param L_Iris (out)left iris data
 * @param R_Iris (out)right iris data
 * @param LR_size (out)length of left and right iris data
 * @param type (in)1:VGA(640:480) 2:K3 3:K7
 * @param pic (in)image save format, 1: BMP, 2: JPG, 3: JP2
 * @return >= 0 Success, -1:Too close, -2:Too far, -5:No iris, -15:Low quality
 */
public int split(byte[] imageData, byte[] L_Iris, byte[] R_Iris, int[] LR_size, int type, Pic
pic)

/**
 * Conver iris image
 *
 * @param imageData (in) Original iris image
 * @param L_Iris (out)left iris data
 * @param R_Iris (out)right iris data
 * @param LR_size (out)length of left and right iris data
 * @param out_lr_score (out) score of left and right iris int[6], 【left iris quality score,
diameter, valid area, right iris quality score, diameter, valid area】
 * @param type (in)1:VGA(640:480) 2:K3 3:K7
 * @param pic (in) image save format, 1: BMP, 2: JPG, 3: JP2
 * @param quality (in)quality score threshold
 * @return >= 0 Success, -1:Too close, -2:Too far, -5:No iris, -15:Low quality
 */
public int split(byte[] imageData, byte[] L_Iris, byte[] R_Iris, int[] LR_size, int[] out_lr_s
core, int type, Pic pic, int quality)

/**
 * Get coordinates of iris
 *
 * @param image image data (NV21 format)
 * @param width image width
 * @param height image height
 * @return
 */
public IrisPosition[] getIrisPosition(byte[] image, int width, int height)

```

3.3.2 Iris Fragment

com.eyecool.fragment.IrisFragment

```
/**
 * Open camera
 */
public void openCamera()

/**
 * Close camera
 */
public synchronized void closeCamera()

/**
 * Get camera data
 */
public byte[] getCameraBytes()

/**
 * Iris enrollment
 *
 * @param timeout      timeout period
 * @param enrollCallback callback interface
 */
public void enroll(final long timeout, final EnrollCallback enrollCallback)

/**
 * Iris verification
 *
 * @param timeout      timeout period
 * @param verifyCallback callback interface
 * @param template      template to be compared
 */
public void verify(final long timeout, final VerifyCallback verifyCallback, final byte[] template)

/**
 * Iris verification (1: N)
 *
 * @param timeout      timeout period
 * @param verifyCallback callback interface
```

```

    * @param templates      list of templates to be compared
    */
    public void verify(final long timeout, final VerifyCallback verifyCallback, final List<byte[]>
templates)

    /**
    * Get iris image

    *
    * @param timeout          timeout period
    * @param quality          quality score
    * @param acquireImageCallback callback interface
    */
    public void acquireImage(final long timeout, final int quality, final AcquireImageCallback acq
uireImageCallback)

    /**
    * Iris verification (1: N)
    *
    * @param timeout          timeout period
    * @param quality          quality score
    * @param imageType        0:K1, 1:VGA (K2), 2:K3, 3:K7
    * @param acquireImageCallback callback interface
    */
    public synchronized void acquireImage(final long timeout, final int quality, int imageType, fi
nal AcquireImageCallback acquireImageCallback)

```

3.4 Error Code

Enrollment and Verification

Error Code	Description
-1	Is in the process of enrollment/verification
-2	Camera hasn't been opened
-3	Timeout
-4	Cancel
-19	Algorithm is not activated

Camera

```
/** Success (no error) */
LIBUSB_SUCCESS = 0,

/** Input/output error */
LIBUSB_ERROR_IO = -1,

/** Invalid parameter */
LIBUSB_ERROR_INVALID_PARAM = -2,

/** Access denied (insufficient permissions) */
LIBUSB_ERROR_ACCESS = -3,

/** No such device (it may have been disconnected) */
LIBUSB_ERROR_NO_DEVICE = -4,

/** Entity not found */
LIBUSB_ERROR_NOT_FOUND = -5,

/** Resource busy */
LIBUSB_ERROR_BUSY = -6,

/** Operation timed out */
LIBUSB_ERROR_TIMEOUT = -7,

/** Overflow */
LIBUSB_ERROR_OVERFLOW = -8,

/** Pipe error */
LIBUSB_ERROR_PIPE = -9,

/** System call interrupted (perhaps due to signal) */
LIBUSB_ERROR_INTERRUPTED = -10,

/** Insufficient memory */
LIBUSB_ERROR_NO_MEM = -11,

/** Operation not supported or unimplemented on this platform */
LIBUSB_ERROR_NOT_SUPPORTED = -12,

/* NB: Remember to update LIBUSB_ERROR_COUNT below as well as the
   message strings in strerror.c when adding new error codes here. */

/** Other error */
```



```
LIBUSB_ERROR_OTHER = -99,
```